Intro to AI 1 - Bonus 2

Lior Vinman Roy Levi

November 2025

1 Hurricane Evacuation - How difficult is the problem?

In this assignment we created an environment simulation that runs a variety of search agents, in order to solve the *Hurricane Evacuation Problem*, now we'll see how difficult the problem really is - in terms of computational complexity.

1.1 HEP definition

Let's define the *Hurricane Evacuation Problem* (HEP) as the following language:

HEP := $\{\langle G, w, s, V_{people}, E_{flooded}, V_{kits}, P, Q, U, K\rangle | \exists$ a sequence S of valid actions starting at node s to visit all V_{people} with total time $\leq K\}$

Where:

- $G = (V, E)$ - Undirected and weighted, world's, graph.
- $w : E(G) \to \mathbb{N}$ - Weight function for world's roads.
- $s \in V(G)$ - A starting position on world's graph.
- $V_{people} \subseteq V(G)$ - A set of goal nodes to visit, has people to evacuate.
- $E_{flooded} \subseteq E(G)$ - A set of flooded edges.
- $V_{kits} \subseteq V(G)$ - A set of nodes with *Amphibian Kit*s.
- P - The Kit slower factor.
- Q - The *Equip*'s time penalty.
- U - The *Unequip*'s time penalty.
- K - upper time bound (total operational and computational cost).

Now, after we've formally defined the problem we could try to analyze it and prove it's complexity class.

1.2 HEP is NP

We can show that HEP is in NP (Non-Deterministic Polynomial).

Theorem 1.1. *Let HEP be the language defined at 1.1, $HEP \in NP$.*

Proof. Let $S = (a_1, a_2, ..., a_n)$ be a certificate for some $h \in HEP$, a sequence of valid actions, which performing them will rescue all people in $h[G]$.
To show that $HEP \in NP$, we need to show a validation algorithm that validates S in polynomial time, let's consider:

Algorithm 1 VALIDATE-HEP-CERTIFICATE(h, S)

```
T_cost ← 0
T_ops ← 0

for action ∈ S do
    if T_ops > LIMIT or action = terminate then
        break
    end if

    if action ∉ {traverse, equip, unequip, no-op} then
        return False
    end if

    T_cost ← T_cost + PERFORM(h, x)
    T_ops ← T_ops + 1
end for

if h[V_people] ≠ ∅ or T_cost > h[K] then
    return False
end if

return True
```

Now we'll proof that the algorithm works and runs in polynomial time.

Lemma 1.2 (Algorithm 1 works)**.** VALIDATE-HEP-CERTIFICATE ***works***.

Proof. We'll prove that the algorithm computes right and terminates:

(a) **Termination**: Given $h[G]$ be a finite graph, S would be a finite sequence of actions (to prevent running forever on an infinite graph, we could run the loop up to a LIMIT of actions) thus, the *for* loop will eventually run over all the elements of S and quit (or quit upon reaching a certain bound of actions). Moreover, the PERFORM() function is an immediate function that just changes the state of h (e.g., popping nodes from V_{people} on traverse), and the other parts of the algorithm are non-blocking validations that also returns an immediate answer that not preventing termination.

(b) **Correctness**: Let's assume that S is a valid sequence of actions that reaches the goal of visit all V_{people} under total time of K. We'll define an invariant set $V_{visited}(k)$ that will hold all the visited nodes, up to the k-th action, We'll proof the algorithm is correct by induction on running over the actions.

(i) Base Case: $k = 0 \Rightarrow V_{visited}(0) = \{h[s]\}$, if we didn't perform any action, we are currently standing on the starting node.

(ii) Induction Hypothesis: Assuming that for a concrete $k \in \mathbb{N}$, $V_{visited}(k)$ contains all the nodes we've visited so far when performed the subsequence of actions: $(a_1, a_2, ..., a_k)$
(not including the rest of the sequence: $(..., a_{k+1}, a_{k+2}, ..., a_n)$),
thus, we've left with: $V_{left}(n) = h[V_{people}] \setminus V_{visited}(n)$ to visit.

(iii) Inductive Step: Let's assume performing the $k+1$-th action.
If $a_{k+1} \in \{\texttt{equip}, \texttt{unequip}, \texttt{no-op}\}$ then $V_{visited}(k+1) = V_{visited}(k)$.
If $a_{k+1} = \texttt{traverse}$ then $V_{left}(k) \subset V_{left}(k+1) = V_{left}(k) \cup \{v_{k+1}\}$, adding a new node that we've visited on the $k+1$-th action - if $v_{l+1} \in h[V_{people}]$ then PERFORM() makes a pop:
$h[V_{people}] = h[V_{people}] \setminus \{v_{k+1}\}$, if S is valid and on the $k+1$-th action $V_{visited}(k+1) \neq h[V_{people}]$ then a_{k+1} cannot be `terminate`.

□

Lemma 1.3 (Algorithm 1 computed in polynomial time)**.** *Given a valid input,* VALIDATE-HEP-CERTIFICATE ***decides in polynomial time****.*

Proof. Let's inspect the algorithm and analyze it's operations:

(i) Lines 1-2: Defining variables, would costs O(1) time and space.

(ii) Lines 4-15: Running a loop over a set upon bound, costs O($min(|S|, \text{LIMIT})$).
Inspecting the operations inside the loop we can see validations that costs O(1), updating variables also costs O(1) (assuming we're not analyzing **bit complexity**) and also running the PERFORM() function costs O(1) (in each iteration it updates the state of h, which is actually just updating h's variables).
Total for the loop: O($min(|S|, \text{LIMIT})$) which equals to O($|S|$), in worst case.

(iii) Lines 17-21: static IF statements with RETURN - are all O(1).

Total for the algorithm: O($|S|$)$\Rightarrow$ polynomial in the size of the input sequence.

□

To sum up, because we've showed a polynomial time verification algorithm for a given certificate, we can tell that $HEP \in NP$, as required. □

1.3 HEP is NPH

Also, we can show that *HEP* is in *NPH* (NP-Hard).

Let's consider the following langauge which is known to be a NPH problem (e.g., according to `Wikipedia`), we'll define it here, by:

$$\text{Path-TSP} := \{\langle G, w, K\rangle | \exists \text{ an hamiltonian path with total cost} \leq K \ \}$$

Where G, w, K are defined as in the *HEP* language (see 1.1).

Theorem 1.4. *Let HEP be the language defined at 1.1, $HEP \in NPH$.*

Proof. We want to show that we can make a polynomial time reduction: $Path - TSP \leq_p HEP$. Let's consider the following gadget:

$$\begin{cases} f : Path - TSP \to HEP \\ f(\langle G, w, K\rangle) = \langle G', w', s_{new}, V(G), \emptyset, \emptyset, 0, 0, 0, K\rangle \end{cases}$$

Where:

- $V'(G) = V(G) \cup \{s_{new}\}$ - Adding a new node.
- $E'(G) = E(G) \cup \{\{s_{new}, v\} | v \in V(G)\}$ - Connecting the new node to all other existing nodes in the graph.
- $w'(e) = \begin{cases} w(e) & \text{if } e \in E(G) \\ 0 & \text{if } v_{new} \in e \end{cases}$ - Making the traverse from new node to all other nodes, be free (cost 0).
- $G' = (V'(G), E'(G))$ - Updating the output graph.

We now want to show three conditions required for the reduction to be valid:

(a) f converts any $Path - TSP$ instance to an HEP instance:
Trivially true - because that's how we've built f.

(b) f could be computed in a polynomial time:
Copying the original graph G would take O($|V(G)| + |E(G)|$), adding the new node O(1), and connecting it to all all existing nodes O($|V(G)|$) placing the 0-s and $\emptyset$-s will take O(1), so it indeed polynomial in the size of G.

(c) f is valid reduction: $x \in Path - TSP \iff f(x) \in HEP$:

  (i) $\Rightarrow$: Let $x \in Path - TSP \Rightarrow \exists$ sequence of nodes:
  $S_x = (v_1, v_2, ..., v_n)$, that constructs an Hamiltonian path that holds: $\sum_{i=1}^{n-1} w(\{v_i, v_{i+1}\}) \leq x[K]$, applying $f(x)$, we'll add a v_{new} which is connected to every other node at cost 0, and since there is an Hamiltonian path in $x[G]$ we can follow it, and thus cover at least $V_{people} \subseteq V(G)$ nodes.

(ii) $\Leftarrow$: Let $f(\langle G, w, K\rangle) \in HEP \Rightarrow \exists$ sequence of actions: $S_{f(x)} = (a_1, a_2, ..., a_n)$ so that if performing all actions we'll cover all nodes in $V(G)$ in total time $\leq K$, giving us a Hamiltonian path: $S' = (s_{new}, v_1, v_2, ..., v_n)$, that implies total cost $\leq K$, traversing the edge $\{s_{new}, v_1\}$ costs 0, so we can pop s_{new} and look on the new sequence: $S'' = (v_1, v_2, ..., v_n)$ which covers $V(G)$ on original graph $G \Rightarrow \langle G, w, K\rangle \in Path - TSP$

$\square$

1.4 HEP is NPC

Theorem 1.5. *Let HEP be the language defined at 1.1, $HEP \in NPC$.*

Proof. $HEP \in NPC \iff HEP \in NP \cap NPH \iff HEP \in NP \wedge HEP \in NPH$.

1. We've proved that $HEP \in NP$ in theorem 1.1.
2. We've proved that $HEP \in NPH$ in theorem 1.5.

combining $1 + 2 \Rightarrow HEP \in NPC$ $\square$